

PURPLE COMPANY – PAYMENT MANAGEMENT SYSTEM

1. Database Design

The Payment entity is designed to store and manage payment transaction details for customer orders in Purple Company. It records information such as Payment ID, Order ID, Payment Method, Payment Status, Payment Date, and Amount. The `Payment_ID` uniquely identifies each payment transaction and acts as the Primary Key. The `Order_ID` connects the Payment table with the Orders table and acts as a Foreign Key. The Payment table helps to manage successful and failed transactions and provides useful information for analyzing payment methods and generating payment transaction reports.

1.1 Payment Table

Attribute	Key	Type
Payment_ID	Primary Key	NUMBER
Order_ID	Foreign Key	NUMBER
Payment_Method	-	VARCHAR2
Payment_Status	-	VARCHAR2
Payment_Date	-	DATE
Amount	-	NUMBER

2. Create Payment Table

The Payment table is created to store payment transaction details associated with Purple Company orders.

```
SQL> CREATE TABLE PURPLE_PAYMENT (  
2     Payment_ID NUMBER PRIMARY KEY,  
3     Order_ID NUMBER,  
4     Payment_Method VARCHAR2(30),  
5     Payment_Status VARCHAR2(20),  
6     Payment_Date DATE,  
7     Amount NUMBER(10,2),  
8     FOREIGN KEY (Order_ID)  
9     REFERENCES PURPLE_ORDERS(Order_ID)  
10 );  
  
Table created.
```

3. Payment Records

Payment records are inserted into the PURPLE_PAYMENT table with payment method, status, date, and amount.

```
SQL> INSERT INTO PURPLE_PAYMENT
  2 (Payment_ID, Order_ID, Payment_Method, Payment_Status, Payment_Date, Amount)
  3 VALUES
  4 (1, 101, 'UPI', 'Successful', '20-SEP-2026', 5000);

1 row created.

SQL>
SQL> INSERT INTO PURPLE_PAYMENT
  2 (Payment_ID, Order_ID, Payment_Method, Payment_Status, Payment_Date, Amount)
  3 VALUES
  4 (2, 102, 'Credit Card', 'Successful', '21-SEP-2026', 1800);

1 row created.

SQL>
SQL> INSERT INTO PURPLE_PAYMENT
  2 (Payment_ID, Order_ID, Payment_Method, Payment_Status, Payment_Date, Amount)
  3 VALUES
  4 (3, 103, 'Debit Card', 'Failed', '22-SEP-2026', 3200);

1 row created.

SQL>
SQL> INSERT INTO PURPLE_PAYMENT
  2 (Payment_ID, Order_ID, Payment_Method, Payment_Status, Payment_Date, Amount)
  3 VALUES
  4 (4, 104, 'Cash', 'Successful', '23-SEP-2026', 1500);

1 row created.

SQL>
SQL> INSERT INTO PURPLE_PAYMENT
  2 (Payment_ID, Order_ID, Payment_Method, Payment_Status, Payment_Date, Amount)
  3 VALUES
  4 (5, 105, 'UPI', 'Failed', '23-SEP-2026', 4500);

1 row created.

SQL>
SQL> INSERT INTO PURPLE_PAYMENT
  2 (Payment_ID, Order_ID, Payment_Method, Payment_Status, Payment_Date, Amount)
  3 VALUES
  4 (6, 106, 'Net Banking', 'Successful', '24-SEP-2026', 2750);

1 row created.

SQL>
SQL> COMMIT;

Commit complete.
```

4. Display Payment Records

The payment records stored in the table are displayed using the SELECT command.

```
SQL> SELECT * FROM PURPLE_PAYMENT;
```

PAYMENT_ID	ORDER_ID	PAYMENT_METHOD	PAYMENT_STATUS
PAYMENT_D	AMOUNT		
1	101	UPI	Successful
20-SEP-26	5000		
2	102	Credit Card	Successful
21-SEP-26	1800		
3	103	Debit Card	Failed
22-SEP-26	3200		
4	104	Cash	Successful
23-SEP-26	1500		
5	105	UPI	Failed
23-SEP-26	4500		
6	106	Net Banking	Successful
24-SEP-26	2750		

5. Successful Transactions

Successful payment transactions are displayed by filtering the Payment_Status as Successful.

```
SQL> SELECT *  
2 FROM PURPLE_PAYMENT  
3 WHERE Payment_Status = 'Successful';
```

PAYMENT_ID	ORDER_ID	PAYMENT_METHOD	PAYMENT_STATUS
PAYMENT_D	AMOUNT		
1	101	UPI	Successful
20-SEP-26	5000		
2	102	Credit Card	Successful
21-SEP-26	1800		
4	104	Cash	Successful
23-SEP-26	1500		
6	106	Net Banking	Successful
24-SEP-26	2750		

6. Failed Transactions

Failed payment transactions are displayed by filtering the Payment_Status as Failed.

```
SQL> SELECT *
  2  FROM PURPLE_PAYMENT
  3  WHERE Payment_Status = 'Failed';
```

PAYMENT_ID	ORDER_ID	PAYMENT_METHOD	PAYMENT_STATUS
3	103	Debit Card	Failed
22-SEP-26	3200		
5	105	UPI	Failed
23-SEP-26	4500		

7. Modify Payment Status

The payment status can be modified when the transaction status changes.

```
SQL> UPDATE PURPLE_PAYMENT
  2  SET Payment_Status = 'Successful'
  3  WHERE Payment_ID = 5;
```

1 row updated.

```
SQL>
```

```
SQL> COMMIT;
```

Commit complete.

8. Analyze Payment Methods

Payment methods used for Purple Company transactions are analyzed to understand the number and amount of transactions through each method.

8.1 Count Transactions by Payment Method

```
SQL> SELECT
  2     Payment_Method,
  3     COUNT(*) AS Total_Transactions
  4 FROM PURPLE_PAYMENT
  5 GROUP BY Payment_Method
  6 ORDER BY Payment_Method;
```

PAYMENT_METHOD	TOTAL_TRANSACTIONS
-----	-----
Cash	1
Credit Card	1
Debit Card	1
Net Banking	1
UPI	2

8.2 Total Amount by Payment Method

```
SQL> SELECT
  2     Payment_Method,
  3     SUM(Amount) AS Total_Amount
  4 FROM PURPLE_PAYMENT
  5 GROUP BY Payment_Method
  6 ORDER BY Payment_Method;
```

PAYMENT_METHOD	TOTAL_AMOUNT
-----	-----
Cash	1500
Credit Card	1800
Debit Card	3200
Net Banking	2750
UPI	9500

9. Payment Transaction Report

A payment transaction report is generated by combining the payment details with the corresponding Purple Company order details.

```

SQL> SELECT
2      p.Payment_ID,
3      p.Order_ID,
4      o.Customer_ID,
5      p.Payment_Method,
6      p.Payment_Status,
7      p.Payment_Date,
8      p.Amount
9  FROM PURPLE_PAYMENT p
10 JOIN PURPLE_ORDERS o
11 ON p.Order_ID = o.Order_ID
12 ORDER BY p.Payment_ID;

```

PAYMENT_ID	ORDER_ID	CUSTOMER_ID	PAYMENT_METHOD
------------	----------	-------------	----------------

PAYMENT_STATUS	PAYMENT_D	AMOUNT
----------------	-----------	--------

1	101	1 UPI
Successful	20-SEP-26	5000
2	102	2 Credit Card
Successful	21-SEP-26	1800
3	103	3 Debit Card
Failed	22-SEP-26	3200

PAYMENT_ID	ORDER_ID	CUSTOMER_ID	PAYMENT_METHOD
------------	----------	-------------	----------------

PAYMENT_STATUS	PAYMENT_D	AMOUNT
----------------	-----------	--------

4	104	4 Cash
Successful	23-SEP-26	1500
5	105	5 UPI
Successful	23-SEP-26	4500
6	106	6 Net Banking
Successful	24-SEP-26	2750